Numerical Computing with NumPy

Lecture 6 · Learning Python at the University of Glasgow

Khiem Nguyen

9 October 2026

Contents

1 From Python lists to NumPy arrays **3**
- 1.1 List comprehensions . . . 3
- 1.2 What is NumPy? . . . 6
- 1.3 Installing and importing NumPy . . . 6
- 1.4 Why not just use lists? . . . 7

2 Creating arrays **8**
- 2.1 From a list: `np.array()` . . . 8
- 2.2 Arrays of zeros, ones and other constants . . . 8
- 2.3 Ranges of numbers: `np.arange()` and `np.linspace()` . . . 9
- 2.4 Data types . . . 10
- 2.5 Random numbers . . . 10

3 Shape, size and dimensions **11**

4 Indexing and slicing **13**
- 4.1 One-dimensional arrays . . . 13
- 4.2 Multi-dimensional arrays . . . 13
- 4.3 Selecting elements with conditions: Boolean masks . . . 15
- 4.4 Selecting elements with lists of indices . . . 16
- 4.5 Views and copies . . . 16

5 Changing the shape and combining arrays **17**
- 5.1 Reshaping . . . 17
- 5.2 Transposing . . . 18
- 5.3 Adding an axis: row and column vectors . . . 19
- 5.4 Flattening . . . 19
- 5.5 Joining arrays . . . 20
- 5.6 Sorting and reversing . . . 21

6 Computing with arrays **22**
- 6.1 Element-wise arithmetic . . . 22
- 6.2 Universal functions . . . 23
- 6.3 Broadcasting . . . 24

6.4 Aggregations: sum, minimum, maximum, mean, ... 25

7 Linear algebra 27
7.1 Dot product of two vectors 27
7.2 Matrix multiplication 27
7.3 Matrix–vector products 28
7.4 Solving systems of linear equations 29

8 Implementing mathematical formulas 29
8.1 Example: the mean squared error 29
8.2 Double sums and `np.einsum()` 30

9 Reading the documentation 32

Summary 32

Exercises 32
Exercise 1: Indexing, slicing and masks 32
Exercise 2: Cleaning sensor data 33
Exercise 3: A week of temperatures 33
Exercise 4: Projectile trajectories with broadcasting 33
Exercise 5: Mesh analysis of a circuit 34
Exercise 6: Lists versus NumPy, revisited 34

Further reading 34

Learning objectives

By the end of this lecture, you should be able to:

- build lists concisely with list comprehensions, and explain why NumPy arrays are faster than lists for numerical work;
- create arrays with `np.array()`, `np.zeros()`, `np.ones()`, `np.arange()`, `np.linspace()` and random-number generators;
- describe an array by its number of dimensions, shape, size and data type;
- access elements with indexing, slicing, Boolean masks and lists of indices, and explain the difference between a *view* and a *copy*;
- reshape, transpose, flatten, join, sort and reverse arrays;
- compute with whole arrays at once, using element-wise arithmetic, universal functions, broadcasting and aggregations along an axis;
- multiply vectors and matrices and solve systems of linear equations;
- translate mathematical formulas, including sums over several indices, into NumPy code.

1 From Python lists to NumPy arrays

1.1 List comprehensions

Before we meet NumPy, let us learn one more useful feature of plain Python lists. **List comprehensions** provide a concise way of creating lists. They are commonly used to build a new list in which each element is the result of an operation applied to each item of another sequence, or to select only the items that satisfy a certain condition.

A list comprehension consists of square brackets containing an expression followed by a `for` clause, and then zero or more further `for` or `if` clauses. The result is a new list, obtained by evaluating the expression for every combination produced by the `for` and `if` clauses. In other words, the loop

```
new_list = []
for item in old_list:
    if condition(item):
        new_list.append(expression(item))
```

can be written in a single line as

```
new_list = [expression(item) for item in old_list if condition(item)]
```

```
# Example 1: the squares of the numbers between 4 and 7
old_list = list(range(10))

new_list = []                          # with an ordinary loop ...
for i in old_list:
    if 3 < i < 8:
        new_list.append(i * i)
print(new_list)

new_list = [i * i for i in old_list if 3 < i < 8]   # ... and with a list
 ↪ comprehension
print(new_list)
```

```
[16, 25, 36, 49]
[16, 25, 36, 49]
```

```
# Example 2: two for clauses and one if clause
pairs = [(x, y) for x in [1, 2, 3] for y in [3, 1, 4] if x != y]

# This is equivalent to the nested loops
combs = []
for x in [1, 2, 3]:
    for y in [3, 1, 4]:
        if x != y:
            combs.append((x, y))

print("With a list comprehension:", pairs)
print("With nested loops:       ", combs)
```

```
With a list comprehension: [(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
With nested loops:         [(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

```
# More examples
squares = [i**2 for i in range(10)]
print("squares =", squares)

lower_case = [s.lower() for s in ["A", "B", "C", "X", "Y", "Z", "ABC"]]
print("lower_case =", lower_case)

text = "Hello World 2 times and 3 times"
```

```
digits_only = [ch for ch in text if ch.isdigit()]    # keep only the
 ↪  characters that are digits
print("digits_only =", digits_only)

sums = [x + y for x in [10, 30, 50] for y in [20, 40, 60]]
print("sums =", sums)
```

```
squares = [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
lower_case = ['a', 'b', 'c', 'x', 'y', 'z', 'abc']
digits_only = ['2', '3']
sums = [30, 50, 70, 50, 70, 90, 70, 90, 110]
```

A list comprehension can even contain another list comprehension, which is a compact way of building a list of lists, i.e. a matrix (Lecture 3):

```
matrix_ones = [[1 for col in range(5)] for row in range(3)]      # 3 rows,
 ↪  5 columns
matrix_mult = [[i * j for j in range(5)] for i in range(6, 10)]  # 4 rows,
 ↪  5 columns

for row in matrix_mult:      # print one row per line
    print(row)
```

```
[0, 6, 12, 18, 24]
[0, 7, 14, 21, 28]
[0, 8, 16, 24, 32]
[0, 9, 18, 27, 36]
```

As a final example, here is the product of a 3×4 matrix $\mathbf{A}$ and a vector $\mathbf{v}$ with 4 elements, $(\mathbf{Av})_i = \sum_j A_{ij} v_j$, written with lists only. We will compute the same product with NumPy in a single, much simpler line later in this lecture.

```
A = [[i * j for j in range(1, 5)] for i in range(1, 4)]      # 3 rows, 4
 ↪  columns
v = [2, 3, 4, 5]                                             # 4 elements
A_dot_v = [sum(A[i][j] * v[j] for j in range(4)) for i in range(3)]
print("A =", A)
print("A v =", A_dot_v)
```

```
A = [[1, 2, 3, 4], [2, 4, 6, 8], [3, 6, 9, 12]]
A v = [40, 80, 120]
```

List comprehensions in a nutshell

1. They are a short and efficient way of making lists.
2. They consist of square brackets containing an expression, a `for` clause, and then zero or more `for`/`if` clauses.
3. They can hold any kind of object, not just numbers.
4. Use them to *build* lists. To repeat an action such as printing, an ordinary `for` loop is clearer.

1.2 What is NumPy?

List comprehensions are convenient, but they are still loops over individual Python objects. For serious numerical work, we need something faster.

NumPy (*Numerical Python*) is an open-source Python library used in almost every field of science and engineering. It is the universal standard for working with numerical data in Python, and it lies at the core of the scientific Python ecosystem. NumPy's interface is used extensively by pandas, SciPy, Matplotlib, scikit-learn, scikit-image and most other packages for data science and scientific computing. NumPy users range from beginners writing their first programs to researchers doing state-of-the-art scientific and industrial research and development.

The heart of NumPy is the **`ndarray`**: a homogeneous, *n*-dimensional array object, with methods that operate on it efficiently. Together with an enormous library of high-level mathematical functions, it makes Python feel very much like MATLAB.

1.3 Installing and importing NumPy

If the statement `import numpy as np` raises an error, NumPy is probably not installed. You can install it from the Anaconda Prompt (or a terminal) with

```
conda install numpy
```

or, from a Jupyter cell, with

```
%pip install numpy
```

By a convention adopted by the whole community, NumPy is always imported under the shorter name `np`. Please follow it, so that anyone reading your code understands it immediately.

```
import numpy as np
```

1.4 Why not just use lists?

A Python list can contain items of different types, whereas all the elements of a NumPy array must have the **same type**. This restriction is precisely what makes arrays so powerful:

- **Speed.** Because every element has the same type, NumPy can store the numbers in one compact block of memory and process them with fast, pre-compiled C code. Operations on whole arrays run without any Python-level loop.
- **Memory.** An array of numbers uses much less memory than a list of the same numbers.
- **Convenience.** Mathematical operations apply to whole arrays at once, so formulas look like mathematics rather than loops.

The following cell computes $\sin(x)$ at one million points, first with lists and then with NumPy. The exact timings depend on your computer, but the difference is always dramatic.

```
import math
import time

n = 1_000_000          # one million points

start = time.perf_counter()
x_list = [2 * math.pi * i / n for i in range(n)]
y_list = [math.sin(x) for x in x_list]
time_list = time.perf_counter() - start

start = time.perf_counter()
x_array = np.linspace(0, 2 * np.pi, n, endpoint=False)    # the same points
y_array = np.sin(x_array)                                  # no loop
 ↪  needed!
time_numpy = time.perf_counter() - start

print(f"Python lists: {time_list:.4f} s")
print(f"NumPy arrays: {time_numpy:.4f} s")
print(f"NumPy is about {time_list / time_numpy:.0f} times faster here.")
```

```
Python lists: 0.0932 s
NumPy arrays: 0.0058 s
NumPy is about 16 times faster here.
```

2 Creating arrays

2.1 From a list: `np.array()`

To create a NumPy array, pass a list (or a list of lists) to `np.array()`:

```
a = np.array([1, 2, 3])            # a 1D array with three elements
b = np.array([[1, 2], [3, 4]])     # a 2D array: a 2 x 2 matrix
print(a)
print(b)
b                                  # the last expression of a cell is
 ↪  displayed with repr()
```

```
[1 2 3]
[[1 2]
 [3 4]]
```

```
array([[1, 2],
       [3, 4]])
```

> **`print(b)` versus `b`**
>
> Recall from Lecture 5 that `print()` uses an object's `__str__()` method, which produces a readable string, whereas Jupyter displays the last expression of a cell with `__repr__()`, which produces an unambiguous representation. For arrays, `repr()` shows the word `array(...)` (and the data type, if it is not the default), so you can see immediately that you are looking at an `ndarray` and not at a list or a string. In these notes, we mostly use `print()` to show several results from one cell.

2.2 Arrays of zeros, ones and other constants

Very often, we need an array of a given size before we know its contents. NumPy provides several functions for this purpose; the size is given by an integer (for a 1D array) or a tuple (for more dimensions).

Function	Creates
`np.zeros(shape)`	an array filled with zeros
`np.ones(shape)`	an array filled with ones
`np.full(shape, value)`	an array filled with `value`
`np.eye(n)`	the $n \times n$ identity matrix
`np.empty(shape)`	an array whose contents are whatever happened to be in memory

```
print(np.zeros(3))
print(np.ones((2, 3)))       # note the tuple (2, 3): 2 rows, 3 columns
print(np.full((2, 2), 7.5))
print(np.eye(3))
```

```
[0. 0. 0.]
[[1. 1. 1.]
 [1. 1. 1.]]
[[7.5 7.5]
 [7.5 7.5]]
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

`np.empty()` does not initialise the values at all, so its contents are unpredictable. Its only advantage is speed; use it only if you will fill every element afterwards.

2.3 Ranges of numbers: `np.arange()` and `np.linspace()`

To create evenly spaced numbers, we use one of two functions:

Function	Produces	Includes `stop`?
`np.arange(start, stop, step)`	numbers from `start` in steps of `step`	no
`np.linspace(start, stop, num)`	`num` evenly spaced numbers from `start` to `stop`	yes

`np.arange()` works just like Python's `range()`, except that the step may be a floating-point number. `np.linspace()` lets you choose the *number* of points instead of the step, which is usually what we want for plotting (Lecture 7).

```
a = np.arange(4)                  # like range(4): 0, 1, 2, 3
b = np.arange(2, 9, 1.5)          # start = 2, stop = 9 (excluded), step =
 ↪  1.5
c = np.linspace(0, 10, num=5)     # 5 evenly spaced numbers from 0 to 10
 ↪  (included)
print("a =", a)
print("b =", b)
print("c =", c)
```

```
a = [0 1 2 3]
b = [2.  3.5 5.  6.5 8. ]
c = [ 0.   2.5  5.   7.5 10. ]
```

> **Prefer `linspace` for non-integer steps**
>
> With a non-integer step, rounding errors can make it unclear whether `np.arange()` includes a value close to `stop`. If you know how many points you need, `np.linspace()` is the safer choice.

2.4 Data types

All the elements of an array have the same **data type**, stored in the attribute `dtype`. NumPy chooses it from the data: integers give `int64` (on most computers), and numbers with a decimal point give `float64`. Functions such as `np.zeros()` and `np.ones()` create `float64` arrays by default. You can choose the type explicitly with the keyword argument `dtype`, or convert an existing array with the method `astype()`.

```
print(np.array([1, 2, 3]).dtype)
print(np.array([1.0, 2, 3]).dtype)        # one float makes the whole array
 ↪  float
print(np.zeros(2).dtype)

x = np.ones((2, 4), dtype=np.int64)       # choose the type explicitly
print(x.dtype)
print(x.astype(float))                     # a converted copy
```

```
int64
float64
float64
int64
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

2.5 Random numbers

Simulations and tests often need random numbers. The recommended way is to create a **random-number generator** and then ask it for numbers. Giving it a fixed `seed` makes the "random" numbers reproducible, which is invaluable when testing code (compare with Exercise 5 of Lecture 2).

```
rng = np.random.default_rng(seed=42)

print(rng.random(3))                       # 3 uniform numbers in [0, 1)
```

```
print(rng.integers(1, 7, size=10))         # 10 dice rolls: integers from 1
 ↪  to 6
print(rng.normal(loc=0.0, scale=1.0, size=4))   # 4 samples of a normal
 ↪  distribution
```

```
[0.77395605 0.43887844 0.85859792]
[1 5 2 1 4 6 5 5 5 5]
[-0.01680116 -0.85304393  0.87939797  0.77779194]
```

3 Shape, size and dimensions

We often work with the same set of numbers arranged in different shapes. For example, a vector in mathematics with n components can be written as a column vector, which is a matrix of shape $(n \times 1)$, or as a row vector, which is a matrix of shape $(1 \times n)$. Each array therefore carries attributes that describe its layout:

- `ndarray.ndim` is the number of **axes** (dimensions) of the array;
- `ndarray.shape` is a tuple of integers giving the number of elements along each axis; for a 2D array with 2 rows and 3 columns, the shape is `(2, 3)`;
- `ndarray.size` is the total number of elements, i.e. the *product* of the numbers in the shape;
- `ndarray.dtype` is the data type of the elements.

The axes are numbered from zero, as everything in Python. For a 2D array, **axis 0 runs down the rows** and **axis 1 runs across the columns**.

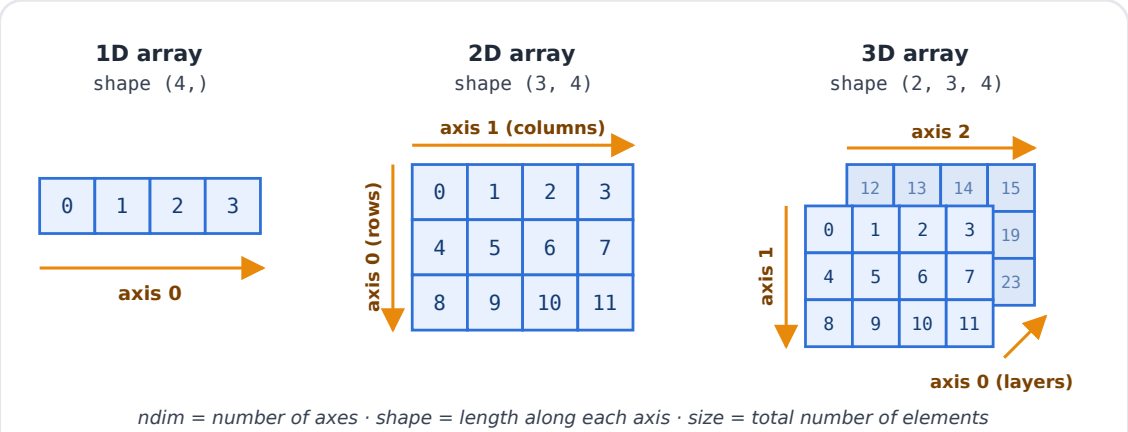


Figure 1: The axes of 1D, 2D and 3D arrays. Axis 0 is always the first index; for a matrix, it counts the rows.

> **A remark on notation**
>
> In these notes, a matrix "of shape $(m \times n)$" — read "m by n" — has m rows and n columns, as is usual in mathematics. In NumPy, the same shape is written as the tuple `(m, n)`.

```
array_2d = np.array([[1, 2, 3],
                     [4, 5, 6]])

print("ndim  =", array_2d.ndim)
print("shape =", array_2d.shape)    # 2 rows, 3 columns
print("size  =", array_2d.size)     # 2 * 3 = 6 elements
print("dtype =", array_2d.dtype)
```

```
ndim  = 2
shape = (2, 3)
size  = 6
dtype = int64
```

```
array_3d = np.array([[[0, 1, 2, 3],
                      [4, 5, 6, 7]],

                     [[0, 1, 2, 3],
                      [4, 5, 6, 7]],

                     [[0, 1, 2, 3],
                      [4, 5, 6, 7]]])

print("ndim  =", array_3d.ndim)
print("shape =", array_3d.shape)
print("size  =", array_3d.size)
print("product of the shape =", np.prod(array_3d.shape))   # 3 * 2 * 4 =
 ↪  size
```

```
ndim  = 3
shape = (3, 2, 4)
size  = 24
product of the shape = 24
```

4 Indexing and slicing

4.1 One-dimensional arrays

We can **index** and **slice** NumPy arrays in the same ways as Python lists (Lecture 3): indices start from zero, negative indices count from the end, and in a slice `start:stop:step` the `stop` index is excluded.

```
data = np.arange(10, 16)       # [10, 11, 12, 13, 14, 15]
print("data       =", data)
print("data[1]    =", data[1])
print("data[-1]   =", data[-1])
print("data[0:2]  =", data[0:2])
print("data[-2:]  =", data[-2:])
print("data[::2]  =", data[::2])
```

```
data       = [10 11 12 13 14 15]
data[1]    = 11
data[-1]   = 15
data[0:2]  = [10 11]
data[-2:]  = [14 15]
data[::2]  = [10 12 14]
```

4.2 Multi-dimensional arrays

For a multi-dimensional array, we give one index or slice **per axis**, separated by commas: `a[i, j]` is the element in row `i` and column `j` of a 2D array. A colon `:` on its own means "everything along this axis". So `a[i, :]` is row `i`, `a[:, j]` is column `j`, and `a[0:2, 1:3]` is a block of the matrix.

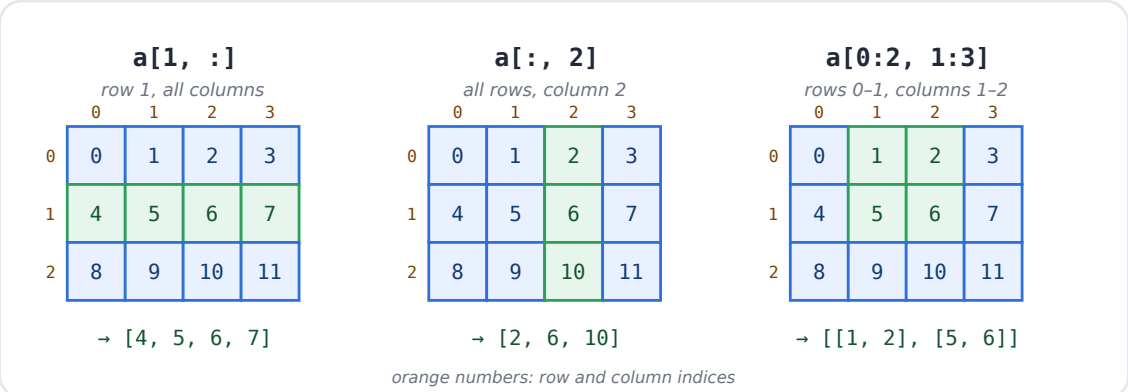


Figure 2: Indexing and slicing a 2D array `a = np.arange(12).reshape(3, 4)`. The highlighted elements are selected.

```
a = np.arange(12).reshape(3, 4)     # reshape() is explained later in this
 ↪  lecture
print(a)
print("a[1, 2]     =", a[1, 2])
print("a[1, :]     =", a[1, :])      # row 1
print("a[:, 2]     =", a[:, 2])      # column 2
print("a[0:2, 1:3] =\n", a[0:2, 1:3])
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
a[1, 2]     = 6
a[1, :]     = [4 5 6 7]
a[:, 2]     = [ 2  6 10]
a[0:2, 1:3] =
 [[1 2]
 [5 6]]
```

```
array_2d = np.array([[1, 2], [3, 4], [5, 6]])

for i in range(array_2d.shape[0]):     # all the rows (along axis 0)
    print("row", i, ":", array_2d[i, :])

for j in range(array_2d.shape[1]):     # all the columns (along axis 1)
    print("column", j, ":", array_2d[:, j])

print(array_3d[0, 0])                  # in 3D, two indices select a whole
 ↪  1D array
```

```
row 0 : [1 2]
row 1 : [3 4]
row 2 : [5 6]
column 0 : [1 3 5]
column 1 : [2 4 6]
[0 1 2 3]
```

`a[i, j]` rather than `a[i][j]`

Both work, but `a[i][j]` first creates the row `a[i]` and then indexes into it. The form `a[i, j]` is faster, and it is the only one that works with slices on several axes, such as `a[:, j]`.

4.3 Selecting elements with conditions: Boolean masks

Selecting the values that satisfy a condition is remarkably simple in NumPy. A comparison such as `a < 5` is evaluated **element by element** and returns an array of Boolean values of the same shape, called a **mask**: `True` where the condition holds and `False` elsewhere. Indexing an array with a mask, `a[mask]`, picks out exactly the elements where the mask is `True`.

To combine conditions, use the operators `&` (and), `|` (or) and `~` (not), with **brackets around each condition**. The Python keywords `and`, `or` and `not` do *not* work element by element, so they cannot be used with arrays.

```
a = np.arange(6)
mask = a < 3
print("a         =", a)
print("mask      =", mask)
print("a[mask]   =", a[mask])
print("a[a >= 4] =", a[a >= 4])                    # the same in one step
print("between   =", a[(a >= 2) & (a <= 4)])        # brackets are
 ↪ essential!
print("outside   =", a[(a < 2) | (a > 4)])
```

```
a         = [0 1 2 3 4 5]
mask      = [ True  True  True False False False]
a[mask]   = [0 1 2]
a[a >= 4] = [4 5]
between   = [2 3 4]
outside   = [0 1 5]
```

```
b = np.arange(1, 6)
print(b[[True, False, True, False, True]])    # a mask can also be written
 ↪ by hand

c = np.array([3.0, -1.5, 2.0, -0.5])
c[c < 0] = 0          # masks can also be used to MODIFY selected elements
print(c)
```

```
[1 3 5]
[3. 0. 2. 0.]
```

```
a = np.arange(6)
a[a >= 2 and a <= 4]  # ValueError: "and" does not work element by element
```

```
ValueError: The truth value of an array with more than one element is ambiguous.
 ↪  Use a.any() or a.all()
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[71], line 2
      1 a = np.arange(6)
----> 2 a[a >= 2 and a <= 4]  # ValueError: "and" does not work element by
 ↪  element
ValueError: The truth value of an array with more than one element is ambiguous.
 ↪  Use a.any() or a.all()
```

4.4 Selecting elements with lists of indices

An array can also be indexed with a **list (or array) of integer indices**, which picks out the elements at those positions in the given order. This is sometimes called *fancy indexing*.

```
x = np.linspace(0, 1, 11)
print(x)
print(x[[0, 5, 10]])       # the first, middle and last elements
print(x[[3, 3, 1]])        # indices may repeat and appear in any order
```

```
[0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1. ]
[0.  0.5 1. ]
[0.3 0.3 0.1]
```

4.5 Views and copies

Here is a crucial difference from Python lists. **Slicing a list creates a copy** (Lecture 3), but **slicing a NumPy array creates a *view***: a new array object that looks at the *same data* as the original. Views save time and memory, because no data are copied — but modifying a view also modifies the original array.

```
a = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
first_row = a[0, :]        # a slice is a VIEW of a
first_row[0] = 99          # changing the view ...
print(a)                   # ... changes the original array!
```

```
[[99  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

To obtain an independent array, make an explicit **copy** with the method `copy()`. The function `np.shares_memory()` tells you whether two arrays share their data.

```
b = a.copy()              # a complete, independent copy
b[0, 0] = 0
print("a[0, 0] =", a[0, 0], "  b[0, 0] =", b[0, 0])
print(np.shares_memory(a, first_row), np.shares_memory(a, b))
```

```
a[0, 0] = 99   b[0, 0] = 0
True False
```

> **!** Rule of thumb
>
> Basic slicing (`a[1:3]`, `a[:, 0]`, ...) returns a **view**. Indexing with a Boolean mask or a list of indices returns a **copy**. When in doubt, and you need an independent array, call `copy()`.

5 Changing the shape and combining arrays

5.1 Reshaping

The method `reshape()` gives an array a new shape **without changing its data**. The new shape must contain **the same number of elements** as the original one. For example, a 1D array of 12 elements can be reshaped into a matrix of shape (2×6), (3×4), (4×3) or (6×2), or into a 3D array of shape $(2 \times 3 \times 2)$, and so on. One of the sizes may be given as `-1`, in which case NumPy works it out for you.

```
a = np.arange(12)
print(a)
print(a.reshape(3, 4))
print(a.reshape(2, 3, 2))       # a 3D array
print(a.reshape(2, -1).shape)   # -1: "work out this size for me"
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[[ 0  1]
  [ 2  3]
  [ 4  5]]

 [[ 6  7]
  [ 8  9]]
```

```
  [10 11]]]
(2, 6)
```

```
a.reshape(5, 3)    # ValueError: 12 elements cannot fill a 5 x 3 array
```

```
ValueError: cannot reshape array of size 12 into shape (5,3)
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[76], line 1
----> 1 a.reshape(5, 3)    # ValueError: 12 elements cannot fill a 5 x 3 array
ValueError: cannot reshape array of size 12 into shape (5,3)
```

The function `np.reshape()` does the same thing; its optional argument `order` decides in which order the elements are read and placed:

```
np.reshape(a, (3, 4), order='C')
```

- `order='C'` (the default) fills the new array **row by row**: the *last* index changes fastest. This is how the C language stores matrices in memory, so C is called a **row-major** language.
- `order='F'` fills the new array **column by column**: the *first* index changes fastest. This is how Fortran (and MATLAB) store matrices, so they are called **column-major** languages.

```
print(np.reshape(a, (3, 4), order='C'))
print(np.reshape(a, (3, 4), order='F'))
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[ 0  3  6  9]
 [ 1  4  7 10]
 [ 2  5  8 11]]
```

5.2 Transposing

Transposing a matrix — swapping its rows and columns — is so common in engineering that arrays have the short attribute `T` for it. The method `transpose()` does the same.

```
matrix = np.arange(1, 7).reshape(2, 3)
print(matrix)
print(matrix.T)
print(matrix.transpose().shape)
```

```
[[1 2 3]
 [4 5 6]]
[[1 4]
 [2 5]
 [3 6]]
(3, 2)
```

5.3 Adding an axis: row and column vectors

Consider

```
a = np.arange(6)
print(a, a.shape)
```

```
[0 1 2 3 4 5] (6,)
```

The shape is `(6,)` rather than `(1, 6)` or `(6, 1)`. Mathematically, `a` is simply a vector with 6 components; it is neither a row nor a column. A **row vector** $\begin{bmatrix}0 & 1 & \cdots & 5\end{bmatrix}$ is a matrix of shape (1×6), and a **column vector** is a matrix of shape (6×1). To turn the vector into one of them, we must add a new axis of length 1, using `np.newaxis` in an index (or, equivalently, `np.expand_dims()`):

```
a_row = a[np.newaxis, :]        # new axis in position 0: shape (1, 6)
a_col = a[:, np.newaxis]        # new axis in position 1: shape (6, 1)
print(a_row, a_row.shape)
print(a_col.shape)

print(np.expand_dims(a, axis=0).shape)    # the same as a[np.newaxis, :]
print(np.expand_dims(a, axis=1).shape)    # the same as a[:, np.newaxis]
```

```
[[0 1 2 3 4 5]] (1, 6)
(6, 1)
(1, 6)
(6, 1)
```

You will see why this matters in the sections on broadcasting and on matrix–vector products.

5.4 Flattening

To turn a multi-dimensional array into a 1D array, use `flatten()` or `ravel()`. The difference is that `flatten()` always returns a **copy**, whereas `ravel()` returns a **view** whenever possible, which saves memory but means that changes affect the original array.

```
x = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
a1 = x.flatten()
a1[0] = 999
print("after changing the flattened copy:\n", x)

a2 = x.ravel()
a2[0] = 999
print("after changing the ravelled view:\n", x)
```

```
after changing the flattened copy:
 [[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
after changing the ravelled view:
 [[999   2   3   4]
 [  5   6   7   8]
 [  9  10  11  12]]
```

5.5 Joining arrays

We often need to join several arrays into a bigger one. Joining two 1D arrays is simple: there is only one way of doing it. For two matrices such as

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix},$$

there are two ways. We can put $\mathbf{B}$ *below* $\mathbf{A}$, along axis 0, to obtain a (4×2) matrix $\mathbf{C}$, or *beside* $\mathbf{A}$, along axis 1, to obtain a (2×4) matrix $\mathbf{D}$:

$$\mathbf{C} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{bmatrix}, \qquad \mathbf{D} = \begin{bmatrix} 1 & 2 & 5 & 6 \\ 3 & 4 & 7 & 8 \end{bmatrix}.$$

The function `np.concatenate()` joins arrays along the axis given by its keyword argument `axis`. For matrices, `np.vstack()` (vertical stack) and `np.hstack()` (horizontal stack) are convenient shortcuts for `axis=0` and `axis=1`. The arrays must have matching sizes along all the *other* axes: a (2×2) and a (4×2) matrix can be stacked vertically (giving 6×2), but not horizontally.

```
print(np.concatenate((np.array([1, 2, 3]), np.array([4, 5]))))   # note the
 ↪  tuple
```

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
C = np.concatenate((A, B), axis=0)    # the same as np.vstack((A, B))
D = np.concatenate((A, B), axis=1)    # the same as np.hstack((A, B))
print("C =\n", C)
print("D =\n", D)
print(np.array_equal(C, np.vstack((A, B))), np.array_equal(D, np.hstack((A,
 ↪  B))))
```

```
[1 2 3 4 5]
C =
 [[1 2]
 [3 4]
 [5 6]
 [7 8]]
D =
 [[1 2 5 6]
 [3 4 7 8]]
True True
```

5.6 Sorting and reversing

`np.sort()` returns a sorted **copy** of an array (in ascending order, by default), while the method `a.sort()` sorts the array **in place**. For multi-dimensional arrays, you can choose the `axis` along which to sort; see the documentation for more options.

`np.flip()` reverses the order of the elements along a given axis — or along all axes, if no axis is given.

```
arr = np.array([2, 1, 5, 3, 7, 4, 6, 8])
print(np.sort(arr))
print(np.flip(arr))
```

```
[1 2 3 4 5 6 7 8]
[8 6 4 7 3 5 1 2]
```

```
arr_2d = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
print("reversed along both axes:\n", np.flip(arr_2d))
print("reversed along axis 0 (the order of the rows):\n", np.flip(arr_2d,
 ↪  axis=0))
```

```
print("reversed along axis 1 (the order of the columns):\n",
 ↪  np.flip(arr_2d, axis=1))
```

```
reversed along both axes:
 [[12 11 10  9]
 [ 8  7  6  5]
 [ 4  3  2  1]]
reversed along axis 0 (the order of the rows):
 [[ 9 10 11 12]
 [ 5  6  7  8]
 [ 1  2  3  4]]
reversed along axis 1 (the order of the columns):
 [[ 4  3  2  1]
 [ 8  7  6  5]
 [12 11 10  9]]
```

We can also reverse a single row or column. Assigning the result back to `arr_2d[1]` writes the reversed values into the second row of `arr_2d` itself:

```
arr_2d[1] = np.flip(arr_2d[1])          # reverse row 1
print(arr_2d)
arr_2d[:, 1] = np.flip(arr_2d[:, 1])    # reverse column 1 (":" means "all
 ↪  the rows")
print(arr_2d)
```

```
[[ 1  2  3  4]
 [ 8  7  6  5]
 [ 9 10 11 12]]
[[ 1 10  3  4]
 [ 8  7  6  5]
 [ 9  2 11 12]]
```

6 Computing with arrays

6.1 Element-wise arithmetic

The most important thing to remember about NumPy is that the arithmetic operators `+`, `-`, `*`, `/` and `**` act **element by element**. If `a` and `b` are two arrays of the same shape and $\circ$ stands for any of these operators, then the result $\mathbf{c} = \mathbf{a} \circ \mathbf{b}$ is the array of the same shape with

$$c_{i_0 i_1 \ldots i_M} = a_{i_0 i_1 \ldots i_M} \circ b_{i_0 i_1 \ldots i_M}$$

for all valid indices. For example, if `a` and `b` have shape `(3, 4)`, then `c = a + b` means `c[i, j] = a[i, j] + b[i, j]` for `i` from 0 to 2 and `j` from 0 to 3. No loop is needed.

```
data = np.array([1, 2])
ones = np.ones(2)
print("data + ones =", data + ones)
print("data - ones =", data - ones)
print("data * ones =", data * ones)    # element by element: NOT a matrix
 ↪  product!
print("data / ones =", data / ones)
print("data ** 2   =", data ** 2)
print("data * 1.6  =", data * 1.6)     # a single number acts on every
 ↪  element
```

```
data + ones = [2. 3.]
data - ones = [0. 1.]
data * ones = [1. 2.]
data / ones = [1. 2.]
data ** 2   = [1 4]
data * 1.6  = [1.6 3.2]
```

6.2 Universal functions

NumPy provides mathematical functions — `np.sqrt()`, `np.exp()`, `np.log()`, `np.sin()`, `np.cos()`, `np.abs()` and many more — that also act element by element on whole arrays. They are called **universal functions** (*ufuncs*). Unlike the functions of the `math` module, which accept only a single number, they work on numbers and arrays alike.

```
x = np.linspace(0, np.pi, 5)
print("x      =", x)
print("sin(x) =", np.sin(x))
print("sqrt   =", np.sqrt(np.array([1.0, 4.0, 9.0])))
print("exp(1) =", np.exp(1.0))         # a single number works too
```

```
x      = [0.         0.78539816 1.57079633 2.35619449 3.14159265]
sin(x) = [0.00000000e+00 7.07106781e-01 1.00000000e+00 7.07106781e-01
 1.22464680e-16]
sqrt   = [1. 2. 3.]
exp(1) = 2.718281828459045
```

6.3 Broadcasting

What happens if the two arrays in an operation do **not** have the same shape? We have already seen that `data * 1.6` multiplies every element by 1.6: NumPy has, in effect, "stretched" the number 1.6 into an array of the same shape as `data`. This mechanism is called **broadcasting**, and it works for arrays too.

NumPy compares the shapes of the two arrays **starting from the last axis**. Two sizes are compatible if they are **equal**, or if **one of them is 1** (or missing). A size of 1 is then stretched to match the other one — virtually, without copying any data.

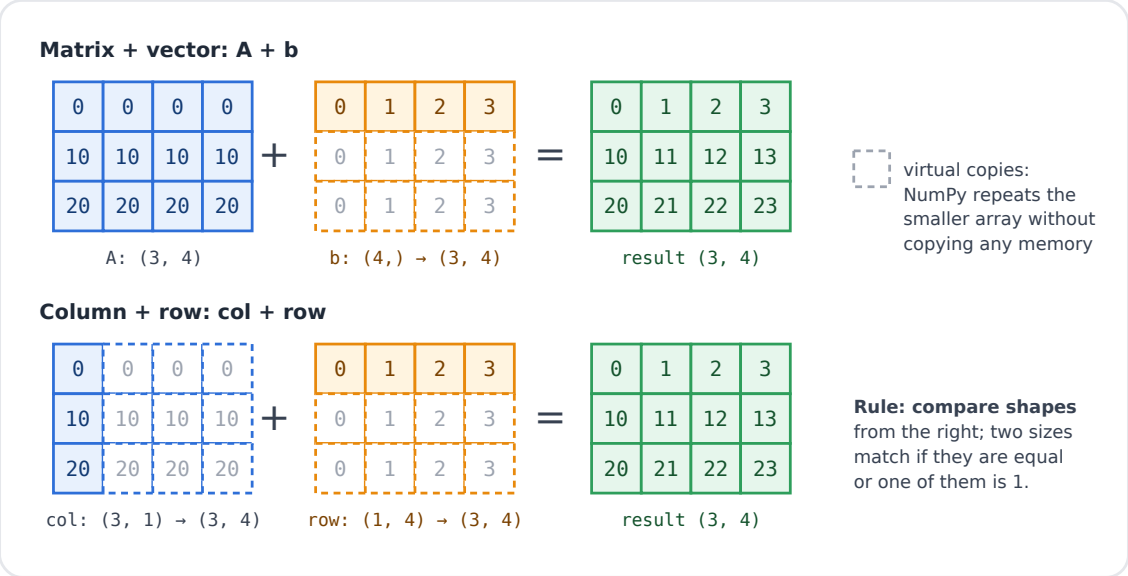


Figure 3: Broadcasting. Top: a vector of shape `(4,)` is added to every row of a `(3, 4)` matrix. Bottom: a column of shape `(3, 1)` and a row of shape `(1, 4)` are both stretched to shape `(3, 4)`. The dashed cells are virtual copies.

```
A = np.array([[0, 0, 0, 0], [10, 10, 10, 10], [20, 20, 20, 20]])
b = np.array([0, 1, 2, 3])
print(A + b)                    # b is added to every row of A

col = np.array([[0], [10], [20]])     # shape (3, 1)
row = np.array([[0, 1, 2, 3]])        # shape (1, 4)
print(col + row)                # both are stretched to shape (3, 4)
```

```
[[ 0  1  2  3]
 [10 11 12 13]
 [20 21 22 23]]
[[ 0  1  2  3]
 [10 11 12 13]
```

```
 [20 21 22 23]]
```

If the shapes are not compatible, NumPy raises an error. Here, the last sizes are 4 and 3, which are neither equal nor 1:

```
M = np.ones((3, 4))
v = np.array([1, 2, 3])
M + v
```

```
ValueError: operands could not be broadcast together with shapes (3,4) (3,)
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[89], line 3
      1 M = np.ones((3, 4))
      2 v = np.array([1, 2, 3])
----> 3 M + v
ValueError: operands could not be broadcast together with shapes (3,4) (3,)
```

To add `v` to every *column* instead, turn it into a column vector of shape `(3, 1)`:

```
print(M + v[:, np.newaxis])
```

```
[[2. 2. 2. 2.]
 [3. 3. 3. 3.]
 [4. 4. 4. 4.]]
```

6.4 Aggregations: sum, minimum, maximum, mean, ...

NumPy arrays have methods that **aggregate** their elements into a single number, such as `sum()`, `prod()`, `min()`, `max()`, `mean()` and `std()` (standard deviation). The methods `argmin()` and `argmax()` return the *position* of the smallest or largest element instead of its value.

```
data = np.array([1, 2, 3, 4])
print("sum =", data.sum(), " prod =", data.prod())
print("min =", data.min(), " max =", data.max())
print("mean =", data.mean(), " std =", round(data.std(), 4))
print("position of the maximum:", data.argmax())
```

```
sum = 10  prod = 24
min = 1  max = 4
mean = 2.5  std = 1.118
position of the maximum: 3
```

It is very common to aggregate along the rows or the columns of a matrix. By default, an aggregation is computed over the *entire* array; with the keyword argument `axis`, it is computed **along that axis only**, which removes the axis from the result. For a matrix, `axis=0` combines the elements *down each column* (giving one result per column), and `axis=1` combines the elements *across each row* (giving one result per row).

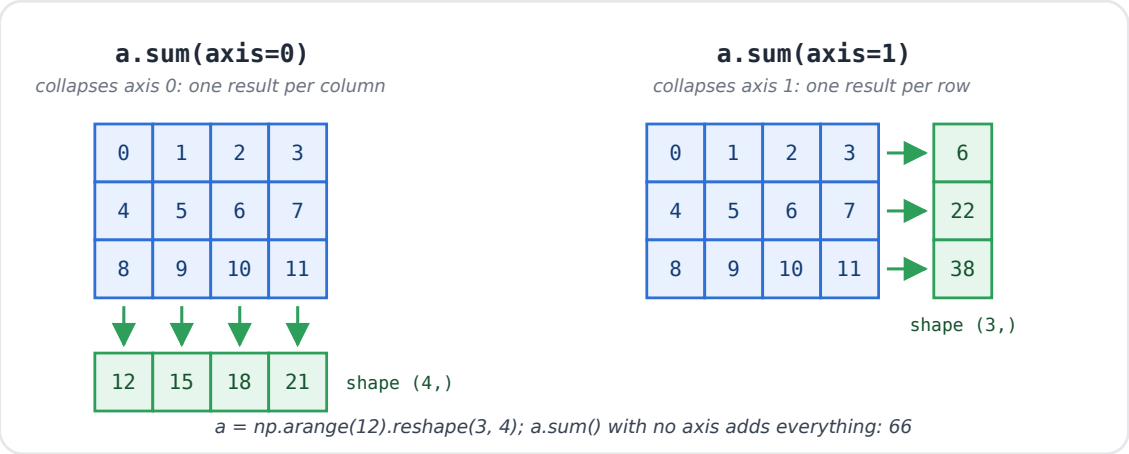


Figure 4: Aggregating along an axis. `axis=0` collapses the rows and gives one sum per column; `axis=1` collapses the columns and gives one sum per row.

```
a = np.array([[11, 10, 1, 12], [9, 6, 5, 7], [2, 4, 3, 8]])
print(a)
print("sum of all elements:  ", a.sum())
print("min of each column:   ", a.min(axis=0))
print("min of each row:      ", a.min(axis=1))
print("mean of each column:  ", a.mean(axis=0))
print("column of each row max:", a.argmax(axis=1))
```

```
[[11 10  1 12]
 [ 9  6  5  7]
 [ 2  4  3  8]]
sum of all elements:   78
min of each column:    [2 4 1 7]
min of each row:       [1 5 2]
mean of each column:   [7.33333333 6.66666667 3.         9.        ]
column of each row max: [3 0 3]
```

```
c = np.arange(12).reshape(2, 3, 2)      # aggregations work in any dimension
print(c.sum(axis=2))                     # sum along the last axis: shape
 ↪  (2, 3)
```

```
[[ 1  5  9]
 [13 17 21]]
```

7 Linear algebra

7.1 Dot product of two vectors

The dot product of two vectors, $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$, can be computed in several equivalent ways:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(np.sum(a * b))     # element-wise product, then sum
print(a.dot(b))          # the method dot()
print(a @ b)             # the matrix-multiplication operator @
```

```
32
32
32
```

7.2 Matrix multiplication

The product $\mathbf{C} = \mathbf{AB}$ of an $(m \times n)$ matrix and an $(n \times p)$ matrix is computed with the operator `@` (or, equivalently, with `A.dot(B)` or `np.matmul(A, B)`).

> ⚠ `*` is not matrix multiplication
>
> `A * B` multiplies two arrays **element by element**. For the matrix product, use `A @ B`. (MATLAB users, beware: in MATLAB, it is exactly the other way round, `.*` versus `*`.)

```
A = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])   # shape (3,
 ↪  4)
B = np.array([[1, 2], [3, 4], [5, 6], [7, 8]])                # shape (4,
 ↪  2)
C = A @ B                                                      # shape (3,
 ↪  2)
print(C)
print(np.array_equal(C, A.dot(B)))
```

```
[[ 50  60]
 [114 140]
 [178 220]]
True
```

7.3 Matrix–vector products

Remember the matrix–vector product that we computed with lists at the start of this lecture? With NumPy, it takes one line:

```
A = np.array([[i * j for j in range(1, 5)] for i in range(1, 4)])   # shape
 ↪  (3, 4)
v = np.array([2, 3, 4, 5])                                          #
 ↪  shape (4,)
print(A @ v)
```

```
[ 40  80 120]
```

The result has shape `(3,)`: a vector, not a column vector of shape `(3, 1)`. This is perfectly consistent with linear algebra: a vector with 3 components is just that, and it is neither a row nor a column (see the section on adding an axis). NumPy treats a 1D array on the right of `@` as whatever is needed to make the product work.

If you explicitly use a column vector of shape `(4, 1)`, the result is a column vector of shape `(3, 1)`. A row vector of shape `(1, 4)`, however, cannot be multiplied from the left by a (3×4) matrix; you must transpose it first:

```
v_col = v[:, np.newaxis]        # shape (4, 1)
v_row = v[np.newaxis, :]        # shape (1, 4)
print((A @ v_col).shape)
print((A @ v_row.T).shape)      # transpose the row vector: (1, 4) -> (4,
 ↪  1)
```

```
(3, 1)
(3, 1)
```

```
A @ v_row       # ValueError: (3 x 4) times (1 x 4) is not defined
```

```
ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0, with
 ↪  gufunc signature (n?,k),(k,m?)->(n?,m?) (size 1 is different from 4)
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[98], line 1
----> 1 A @ v_row       # ValueError: (3 x 4) times (1 x 4) is not defined
ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0, with
 ↪  gufunc signature (n?,k),(k,m?)->(n?,m?) (size 1 is different from 4)
```

7.4 Solving systems of linear equations

Engineering problems constantly lead to systems of linear equations $\mathbf{A}\mathbf{x} = \mathbf{b}$, for example in structural analysis, circuit analysis and the finite element method. The submodule `np.linalg` solves them with `np.linalg.solve()`. For instance, the system

$$\begin{aligned} 2x + y - z &= 8, \\ -3x - y + 2z &= -11, \\ -2x + y + 2z &= -3 \end{aligned}$$

is solved by:

```
A = np.array([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]], dtype=float)
b = np.array([8, -11, -3], dtype=float)
x = np.linalg.solve(A, b)
print("x =", x)
print("check:", np.allclose(A @ x, b))   # True if A x equals b up to
 ↪  rounding errors
```

```
x = [ 2.  3. -1.]
check: True
```

`np.linalg` also provides, among others, `np.linalg.det()` (determinant), `np.linalg.inv()` (inverse) and `np.linalg.eig()` (eigenvalues and eigenvectors). To solve a system, prefer `solve()` to computing the inverse: it is faster and more accurate.

> **Comparing floating-point numbers**
>
> Because of rounding errors, never test floating-point results with `==`. Use `np.allclose()` for arrays, or `np.isclose()` for single numbers.

8 Implementing mathematical formulas

The ease with which mathematical formulas can be translated into array code is one of the main reasons why NumPy is so widely used in science and engineering.

8.1 Example: the mean squared error

The **mean squared error** measures how far the predictions of a model are from measured data. It is central to regression and to machine learning:

$$\text{MSE} = \frac{1}{n}\sum_{j=1}^{n}\left(y_j^{\text{prediction}} - y_j^{\text{data}}\right)^2.$$

In NumPy, the formula reads almost exactly like the mathematics:

```
predictions = np.array([1.0, 1.0, 1.0])
labels = np.array([1.0, 2.0, 3.0])
n = labels.size

error = (1 / n) * np.sum(np.square(predictions - labels))
print(error)                                    # (0 + 1 + 4) / 3
print(np.mean((predictions - labels) ** 2))     # an equivalent one-liner
```

```
1.6666666666666665
1.6666666666666667
```

The arrays `predictions` and `labels` could just as well contain a million values: they only need to have the same shape. The subtraction and squaring happen element by element, and the sum then reduces the result to a single number.

8.2 Double sums and `np.einsum()`

Let **A** and **B** be two matrices of the same size $(m \times n)$. We want to compute the double sum

$$\sigma = \sum_{i=1}^{m}\sum_{j=1}^{n} A_{ij}B_{ij},$$

which appears, for example, in continuum mechanics as the *double contraction* $\mathbf{A} : \mathbf{B}$ of two second-order tensors.

Method 1. Multiply the two matrices element by element, $C_{ij} = A_{ij}B_{ij}$, and then add up all the elements of **C**.

Method 2. Use the function `np.einsum()`. Its first argument is a string that describes the formula with **index letters**: every index letter that appears in the inputs but not in the output is summed over. Here, `'ij,ij'` means "multiply A_{ij} by B_{ij} and sum over both i and j". The name comes from the **Einstein summation convention**, introduced by Albert Einstein, in which a sum over repeated indices is implied rather than written out.

```
A = np.arange(9).reshape(3, 3)
B = np.arange(10, 19).reshape(3, 3)
```

```
print("Method 1:", (A * B).sum())
print("Method 2:", np.einsum('ij,ij', A, B))
```

```
Method 1: 564
Method 2: 564
```

A more advanced example. Let **A** be a four-dimensional array of size $(n_1 \times n_2 \times n_3 \times n_4)$ with components A_{ijkl}, and **B** a matrix of size $(n_1 \times n_2)$ with components B_{ij}. We want the matrix **C** of size $(n_3 \times n_4)$ defined by

$$C_{kl} = \sum_{i=1}^{n_1} \sum_{j=1}^{n_2} A_{ijkl}\, B_{ij} \qquad \text{for all } k = 1, \ldots, n_3 \text{ and } l = 1, \ldots, n_4.$$

For each fixed pair (k, l), this is exactly the double sum of the previous example, so we could compute **C** with two loops over k and l. NumPy offers two loop-free alternatives: `np.einsum()`, where `'ijkl,ij->kl'` names the indices of the output explicitly, and broadcasting, where we give **B** two extra axes of length 1 so that it is stretched along k and l.

```
A = np.arange(3 * 3 * 3 * 3).reshape(3, 3, 3, 3)
B = np.arange(3 * 3).reshape(3, 3)

# Method 1: loops over k and l (easy to understand, but slow for large
 ↪  arrays)
C1 = np.empty((3, 3))
for k in range(3):
    for l in range(3):
        C1[k, l] = (A[:, :, k, l] * B).sum()

# Method 2: Einstein summation
C2 = np.einsum('ijkl,ij->kl', A, B)

# Method 3: broadcasting. B gets shape (3, 3, 1, 1), then we sum over axes
 ↪  0 and 1
C3 = (A * B[:, :, np.newaxis, np.newaxis]).sum(axis=(0, 1))

print(C2)
print(np.allclose(C1, C2), np.allclose(C2, C3))
```

```
[[1836 1872 1908]
 [1944 1980 2016]
 [2052 2088 2124]]
True True
```

9 Reading the documentation

NumPy has hundreds of functions, and nobody remembers all their options. Learn to look them up instead:

```
np.linspace?       # in Jupyter: show the documentation of np.linspace
help(np.sum)       # works everywhere, also in plain Python scripts
```

The online NumPy documentation contains a user guide, many examples, and a complete reference of every function. If you do not understand the formal description of the parameters at first, scroll down to the examples at the end of each page.

Summary

- List comprehensions build lists in one line, but for numerical work, NumPy arrays are far faster and more convenient.
- Create arrays with `np.array()`, `np.zeros()`, `np.ones()`, `np.full()`, `np.eye()`, `np.arange()` (stop excluded), `np.linspace()` (stop included), and random-number generators from `np.random.default_rng()`.
- Every array has a `shape`, a number of dimensions `ndim`, a `size` and a `dtype`. Axis 0 is the first index; for a matrix, it counts the rows.
- Index with `a[i, j]`, slice with `a[start:stop:step, ...]`, select with Boolean masks (combined with `&`, `|`, `~`) or lists of indices. Slices are **views**; use `copy()` for an independent array.
- `reshape()`, `.T`, `np.newaxis`, `flatten()`/`ravel()`, `np.concatenate()`, `np.vstack()`, `np.hstack()`, `np.sort()` and `np.flip()` change the shape or order of arrays.
- Arithmetic and universal functions act element by element; **broadcasting** stretches axes of size 1 to make shapes compatible. Aggregations such as `sum()`, `mean()` and `max()` accept an `axis` argument.
- `@` is the matrix product (`*` is element-wise). Solve linear systems with `np.linalg.solve()`, and compare floating-point results with `np.allclose()`.

Exercises

Exercise 1: Indexing, slicing and masks

Create the array `a = np.arange(1, 37).reshape(6, 6)`. Using indexing and slicing only (no loops), extract and print:

1. the third row and the last column;
2. the central 2×2 block;

3. every other row, starting with the first;
4. the four corner elements, using a list of indices or several slices;
5. all the even numbers, using a Boolean mask.

Finally, set all the multiples of 5 in `a` to zero, and use `np.shares_memory()` to check which of the arrays you extracted above are views of `a`.

Exercise 2: Cleaning sensor data

A sensor stores the value `-999.0` whenever a measurement fails:

```
readings = np.array([12.1, 11.8, -999.0, 12.5, 13.0, -999.0, 12.9, 12.4])
```

Using Boolean masks, compute (a) the number of failed measurements, (b) the mean of the valid readings, and (c) a cleaned copy of `readings` in which each failed value is replaced by that mean. Make sure that the original array `readings` is not modified.

Exercise 3: A week of temperatures

The following code simulates hourly temperatures (in °C) for one week, stored as an array of shape `(7, 24)`: one row per day, one column per hour.

```
rng = np.random.default_rng(seed=1)
temps = 12 + 6 * rng.random((7, 24))
```

Without writing any loop, compute and print with two decimal places:

1. the mean, minimum and maximum temperature of each day;
2. the mean temperature of each hour of the day, averaged over the week;
3. the hour at which the maximum temperature occurs on each day (use `argmax()`);
4. the total number of hours with a temperature above 16 °C.

> **Hint**
>
> Decide for each question whether you need `axis=0`, `axis=1` or no axis at all. For the last question, recall that `True` counts as 1 when you add Boolean values.

Exercise 4: Projectile trajectories with broadcasting

A projectile is launched with speed $v_0 = 20$ m/s at the angles $\theta = 15°, 30°, 45°, 60°$ and $75°$. Its height at time t is $y(t) = v_0 \sin\theta \, t - \tfrac{1}{2} g t^2$, with $g = 9.81\,\mathrm{m/s^2}$.

1. Create an array `theta` of the five angles (in radians) and an array `t = np.linspace(0, 4, 401)` of times.
2. Using broadcasting — and no loop — compute a 2D array `y` of shape `(5, 401)` in which row `i` contains the heights for the angle `theta[i]`.
3. Find the maximum height for each angle with `max(axis=1)`, and compare it with the exact value $(v_0 \sin\theta)^2/(2g)$ using `np.allclose()`. Why are the two slightly different?

(We will plot these trajectories in Lecture 7.)

Exercise 5: Mesh analysis of a circuit

Mesh analysis of a resistor network gives the following equations for the three mesh currents I_1, I_2 and I_3 (in amperes):

$$\begin{aligned} 10I_1 - 4I_2 &= 12, \\ -4I_1 + 12I_2 - 6I_3 &= 0, \\ -6I_2 + 10I_3 &= 6. \end{aligned}$$

Write the system in the form $\mathbf{A}\mathbf{I} = \mathbf{b}$, solve it with `np.linalg.solve()`, and verify your solution with `np.allclose()`. Then compute the determinant of $\mathbf{A}$ with `np.linalg.det()`. What would it mean if the determinant were zero?

Exercise 6: Lists versus NumPy, revisited

In Exercise 4 of Lecture 3, you multiplied two matrices stored as lists of lists. Generate two random matrices of shape (100×100) with `rng.random((100, 100))`, and use `time.perf_counter()` to compare the time taken by your list-based code (after converting the arrays with `tolist()`) with the time taken by `A @ B`. Check with `np.allclose()` that both methods give the same result.

Further reading

- NumPy: the absolute basics for beginners, from the official NumPy documentation.
- Broadcasting and Copies and views, in the NumPy user guide.
- The official Python tutorial: List Comprehensions.